



Practica 7:

PERCEPTRÓN SIMPLE VS. MULTICAPA

Inteligencia Artificial

Profra. Vanessa Alejandra Camacho Vázquez

Grupo: **6CM3**

Nombres de los integrantes:

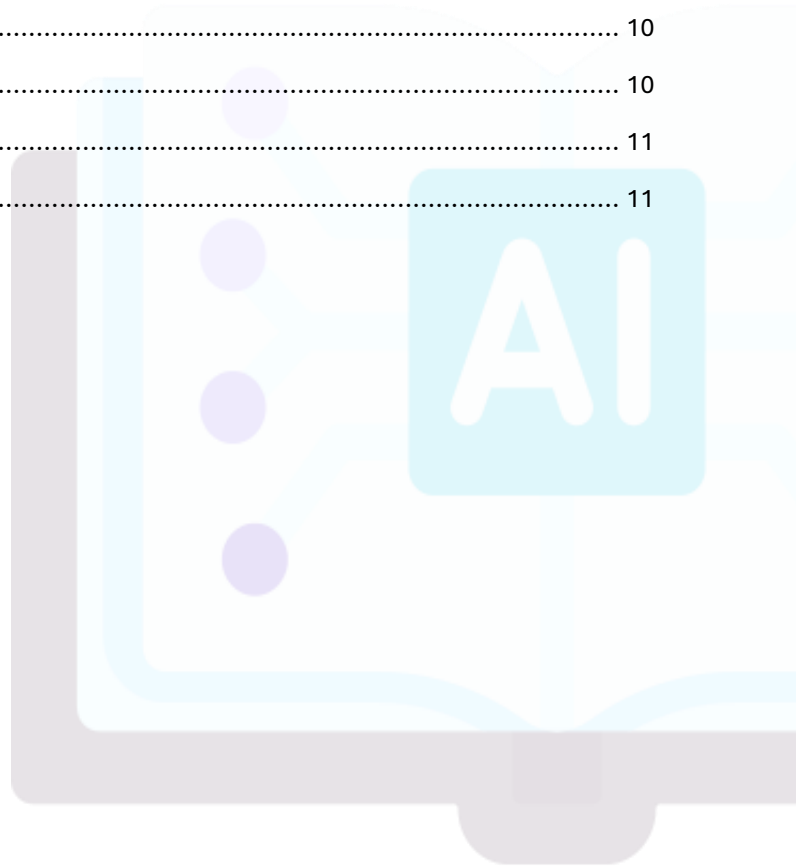
- Aguirre Lanto Víctor Manuel
- Gasca Fragoso Pedro
- Guevara Badillo Areli Alejandra
- Montiel Toro Arael de Jesús
- Ramírez Lozano Gael Martin

Notebook: [Practica no.7](#)

viernes, 21 de noviembre de 2025

CONTENIDO

PROCEDIMIENTO	3
CONFIGURACIÓN DEL ENTORNO Y GENERACIÓN DE DATOS	3
DISEÑO Y ARQUITECTURA DEL MODELO	5
¿Por qué se eligió esa función de pérdida y ese optimizador para minimizar dicha pérdida?	6
Inspección de la estructura	6
EJECUCIÓN DEL ENTRENAMIENTO Y PARÁMETROS	7
VALIDACIÓN Y ANÁLISIS DE RESULTADOS	8
Función de Activación por Defecto	10
CONCLUSIONES	10
AGUIRRE LANTO VÍCTOR MANUEL	10
GASCA FRAGOSO PEDRO	10
GUEVARA BADILLO ARELI ALEJANDRA	10
MONTIEL TORO ARAEL DE JESÚS	11
RAMÍREZ LOZANO GAELE MARTÍN	11



PROCEDIMIENTO

Configuración del entorno y generación de datos

Esta parte del código se encarga de preparar el entorno de trabajo. Se importan las librerías esenciales: TensorFlow y Keras para construir y gestionar el modelo de Deep Learning, y la utilidad `plot_model` para poder visualizar la estructura del modelo. También se integra NumPy (`np`) para las operaciones numéricas eficientes en el manejo de datos, y Matplotlib (`plt`) para generar cualquier tipo de gráfico o visualización de resultados durante o después del proceso.

```
# Importación de bibliotecas
import tensorflow as tf
from tensorflow import keras

from tensorflow.keras.utils import plot_model

import numpy as np
import matplotlib.pyplot as plt
```

Necesitamos crear el conjunto de datos necesario para el ejercicio. Se genera la variable de entrada, llamada `x`, con 50 puntos distribuidos de forma uniforme entre los valores -10 y +10 utilizando la función `linspace` de NumPy. La variable de salida, `y`, se calcula a partir de `x` mediante una relación de segundo grado, a la cual se le añade un factor de ruido aleatorio. Este ruido se genera con la función `np.random.normal`, donde `normal(0, 4)` indica que los valores aleatorios deben seguir una distribución normal con una media de cero y una desviación estándar de cuatro, mientras que el parámetro `size` se utiliza para asegurar que se cree la misma cantidad de puntos que en el arreglo `x`. Finalmente, se muestran en pantalla los valores generados para ambas variables.

```
# Genera datos espaciados uniformemente
x = np.linspace(-10, 10, 50)

# Modelo generado a partir de las entradas
y = 2 * x ** 2 + 3 * x + 5 * np.random.normal(0, 4, size = len(x))

print(f"x: {x}\n")
print(f"y: {y}")
```

El resultado en pantalla es la impresión completa de los dos arreglos generados, x y y. En x se confirman los 50 valores que inician en -10 y terminan en 10, distribuidos uniformemente, verificando la correcta ejecución de np.linspace.

```
x: [-10.      -9.59183673 -9.18367347 -8.7755102  -8.36734694
  -7.95918367 -7.55102041 -7.14285714 -6.73469388 -6.32653061
  -5.91836735 -5.51020408 -5.10204082 -4.69387755 -4.28571429
  -3.87755102 -3.46938776 -3.06122449 -2.65306122 -2.24489796
  -1.83673469 -1.42857143 -1.02040816 -0.6122449  -0.20408163
   0.20408163  0.6122449   1.02040816  1.42857143  1.83673469
   2.24489796  2.65306122  3.06122449  3.46938776  3.87755102
   4.28571429  4.69387755  5.10204082  5.51020408  5.91836735
   6.32653061  6.73469388  7.14285714  7.55102041  7.95918367
   8.36734694  8.7755102   9.18367347  9.59183673 10.      ]

y: [161.82763362 166.67202661 107.25214536 121.46937959 116.61899001
 111.20131495  61.62615265 106.39477093 102.23294504  74.77726305
  48.52876255  36.36543447  44.62878248  43.84622011  16.19085851
  21.53139741 -9.4137247   12.2300189   38.90436301 -0.35962769
 -9.71872077 -5.20164814  4.96011714 -17.69654933  29.91163153
  37.90279254 -39.57901949  9.67138881  5.68825398 -7.80365226
  10.75358584  16.85282231 -9.93514456  59.2758717  52.60359749
  26.48304899  53.20601736  91.61259512  66.82497686 107.40921729
  90.41469322  54.44373364 147.70827336 155.3633152 185.18197533
 154.84717054 187.42046705 172.02983731 195.17571918 240.47367875]
```

Para visualizar los datos, tenemos que definir el tamaño del lienzo y asignar un título general, para luego trazar los puntos de las variables x y y utilizando marcadores circulares que permiten ver la dispersión individual de cada dato. También vamos a configurar las etiquetas de los ejes como Tiempo y Valor y añadimos una leyenda identificativa y una cuadrícula de fondo para facilitar la lectura, por último, plt.grid() para mostrar la gráfica en pantalla.

```
# Generar lienzo
plt.figure(figsize = (15, 8))

#Título del lienzo
plt.title("Vizualizacion de Datos", fontsize = 15)

#Dibujar datos
plt.plot(x, y, 'o' )

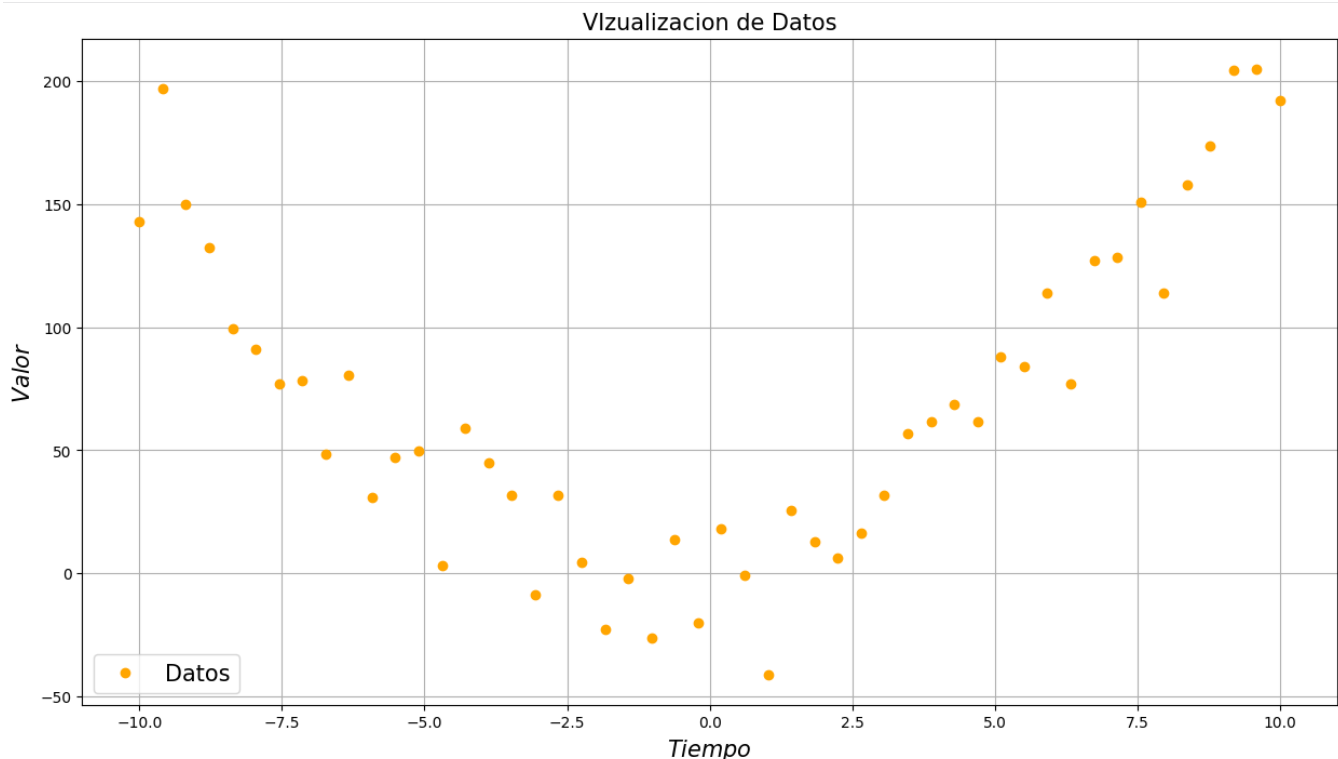
# Leyenda de Los datos
plt.legend(["Datos"], fontsize=15)

# Nombrar Ejes
plt.xlabel("$Tiempo$", fontsize=15)
plt.ylabel("$Valor$", fontsize=15)

# Poner cuadrícula
plt.grid()

# Mostrar imagen
plt.show()
```

En la gráfica se puede observar un conjunto de puntos que siguen una tendencia curva en forma de "U" (parábola), lo cual confirma visualmente la relación cuadrática que definimos en la ecuación. Sin embargo, los puntos no forman una línea perfecta, sino que están dispersos alrededor de esa trayectoria ideal; esta dispersión es la representación visual del ruido aleatorio que introducimos para simular la imperfección de los datos reales.



Diseño y Arquitectura del Modelo

En esta línea de código se define la arquitectura de la red neuronal que vamos a utilizar. Se crea un modelo de tipo Secuencial, lo cual indica que vamos a apilar capas una tras otra de forma lineal. Dentro de este modelo, añadimos una única capa "Densa" (o totalmente conectada) que contiene una sola neurona (`units=1`). Además, configuramos esta capa para que acepte un único valor de entrada (`input_shape=(1,)`), lo que resulta en la estructura más simple posible, equivalente a una regresión lineal, lista para intentar aprender la relación entre los datos.

```
# Creamos un modelo secuencial cuya capa densa tiene una sola neurona unidimensional.
model = keras.Sequential([keras.layers.Dense(units=1, input_shape=(1,))])
```

En esta línea de código completamos la configuración previa al entrenamiento mediante la función `compile`. Aquí definimos dos aspectos cruciales: el optimizador, seleccionado como `'sgd'` (Descenso de Gradiente Estocástico), que será el algoritmo matemático encargado de ajustar los "pesos" o parámetros de la neurona para que aprenda, y la función de pérdida (loss), establecida como `'mean_squared_error'` (Error Cuadrático Medio), la cual servirá como una métrica para calcular qué tan lejos está la predicción del modelo del valor real, indicándole al sistema cuánto debe corregirse en cada paso.

```
# Compilamos el modelo, especificando el optimizador y la función de pérdida.  
model.compile(optimizer='sgd', loss='mean_squared_error')
```

¿POR QUÉ SE ELIGIÓ ESA FUNCIÓN DE PÉRDIDA Y ESE OPTIMIZADOR PARA MINIMIZAR DICHA PÉRDIDA?

Para la función de pérdida se eligió el `mean_squared_error` (Error Cuadrático Medio) debido a que estamos ante un problema de regresión, donde el objetivo es predecir un valor numérico específico. Mide la distancia entre la predicción del modelo y el valor real, elevando esa diferencia al cuadrado. Esto sirve para que el modelo identifique claramente los errores grandes y se vea forzado a corregirlos, buscando siempre acercarse lo más posible al resultado correcto.

Por otro lado, se seleccionó `sgd` (Descenso de Gradiente Estocástico) como el optimizador encargado de reducir esa pérdida. Este algoritmo actúa como el mecanismo de aprendizaje que ajusta paso a paso los parámetros internos de la neurona basándose en el error calculado. Realiza cambios graduales y constantes en los pesos, permitiendo que la red descienda de manera estable hasta encontrar el punto de mínimo error.

INSPECCIÓN DE LA ESTRUCTURA

En este bloque se utiliza la función `model.summary()` para inspeccionar la configuración interna del perceptrón que acabamos de crear. El resultado desplegado en la consola es una tabla técnica que confirma la estructura del modelo, listando nuestra capa densa y especificando que existen exactamente 2 parámetros entrenables; estos valores representan el peso y el sesgo que la red ajustará matemáticamente. Además, la tabla verifica que la forma de los datos de salida es unitaria ((None, 1)), asegurando que la arquitectura es correcta antes de proceder.

```
# Imprimiendo el resumen del perceptron  
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 1)	2

Total params: 2 (8.00 B)

Trainable params: 2 (8.00 B)

Non-trainable params: 0 (0.00 B)

Aquí utilizamos la herramienta `plot_model`, habilitando la opción `show_shapes=True`, para generar un diagrama visual que nos permite ver gráficamente cómo fluyen los datos por nuestro perceptrón. El resultado es una imagen esquemática que representa la capa Densa y confirma explícitamente las dimensiones de trabajo: muestra una entrada de tamaño 1 y una salida de tamaño 1 (denotadas como `(None, 1)`). Esto sirve como una comprobación final y muy intuitiva para asegurar que la red está diseñada correctamente para procesar un solo valor numérico y devolver una única predicción.

```
# Visualizacion del modelo en forma de grafo
plot_model(model, show_shapes=True)
```

Dense

Input shape: **(None, 1)**

Output shape: **(None, 1)**

Ejecución del Entrenamiento y Parámetros

En esta etapa se ejecuta el proceso de aprendizaje del perceptrón utilizando el comando `model.fit`. Aquí se le entregan los datos de entrada `x` y los objetivos `y` y para que la red comience a ajustar sus parámetros, configurando el entrenamiento para que realice 10 iteraciones completas (o epochs) sobre todo el conjunto de datos. El parámetro `verbose=1` es el responsable de mostrar la barra de progreso y los detalles paso a paso en la consola. El resultado visualiza el desempeño en cada una de las 10 iteraciones; lo fundamental es observar la columna `loss`, que indica el error del modelo: este comienza con un valor alto (alrededor de 9542) y desciende gradualmente hasta 6476, lo que demuestra que el modelo está reduciendo su margen de error y empezando a comprender la relación entre los datos.

```
model.fit(x, y, epochs=10, verbose=1)

Epoch 1/10
2/2 ————— 1s 29ms/step - loss: 9542.9023
Epoch 2/10
2/2 ————— 0s 25ms/step - loss: 8122.7153
Epoch 3/10
2/2 ————— 0s 25ms/step - loss: 8131.8105
Epoch 4/10
2/2 ————— 0s 24ms/step - loss: 7023.6313
Epoch 5/10
2/2 ————— 0s 23ms/step - loss: 7468.1299
Epoch 6/10
2/2 ————— 0s 27ms/step - loss: 6675.9897
Epoch 7/10
2/2 ————— 0s 43ms/step - loss: 7024.8428
Epoch 8/10
2/2 ————— 0s 42ms/step - loss: 5973.5859
Epoch 9/10
2/2 ————— 0s 39ms/step - loss: 6508.3145
Epoch 10/10
2/2 ————— 0s 37ms/step - loss: 6476.1406
<keras.src.callbacks.history.History at 0x7a4640f93800>
```

En este fragmento de código recuperamos y organizamos los valores internos que el modelo ha aprendido después de entrenar. Para ello, se utiliza la función `model.get_weights()`, guardando la información en la variable `w`. Posteriormente, colocamos varias líneas de impresión diseñadas para mostrar, paso a paso: el objeto completo con todos los datos, el conteo de cuántos tipos de pesos existe y, finalmente, se extraen y etiquetan por separado el valor exacto del peso (`w`) y del sesgo (`b`) para poder identificarlos con claridad.

```
#Obtener Pesos de La Red
w = model.get_weights();
print(w)

#Imprimir resultados
print("\nObjeto Pesos:",w)
print('\nNumber of Weights -> ' + str(len(w)))
print('\nw = ' + str(w[0][0]) + '(Weight)')
print('b = ' + str(w[1]) + '("Weight"->Bias)')
```

La salida en consola revela los parámetros exactos que el perceptrón ha calculado tras finalizar el entrenamiento. Se confirman dos valores principales almacenados en los arreglos: primero, el peso (`w`) que ha convergido en un valor aproximado de 2.07, y segundo, el sesgo o bias (`b`) que se ha establecido en 23.49. Estos números son fundamentales porque representan la ecuación lineal específica ($y = 2.07x + 23.49$) que según para el modelo, es la mejor para describir los datos y que se utilizará de ahora en adelante para realizar predicciones.

```
[array([[2.0731208]], dtype=float32), array([23.494589], dtype=float32)]
Objeto Pesos: [array([[2.0731208]], dtype=float32), array([23.494589], dtype=float32)]
Number of Weights -> 2
w =[2.0731208](Weight)
b =[23.494589]("Weight"->Bias)
```

Validación y Análisis de Resultados

Ahora podemos poner a prueba el modelo ya entrenado utilizando la función `model.predict(x)`. La red toma todos los valores de entrada originales (`x`) y les aplica la fórmula que acaba de aprender (usando el peso y el sesgo calculados anteriormente) para estimar sus propios resultados de salida. Estos nuevos valores calculados se almacenan en la variable `predecir` y son fundamentales para que, en el siguiente paso, podamos comparar visualmente qué tan bien se ajusta la "línea" que propone el modelo frente a los datos reales. La barra de carga inferior simplemente confirma que el cálculo se ejecutó correctamente sobre todo el conjunto de datos.

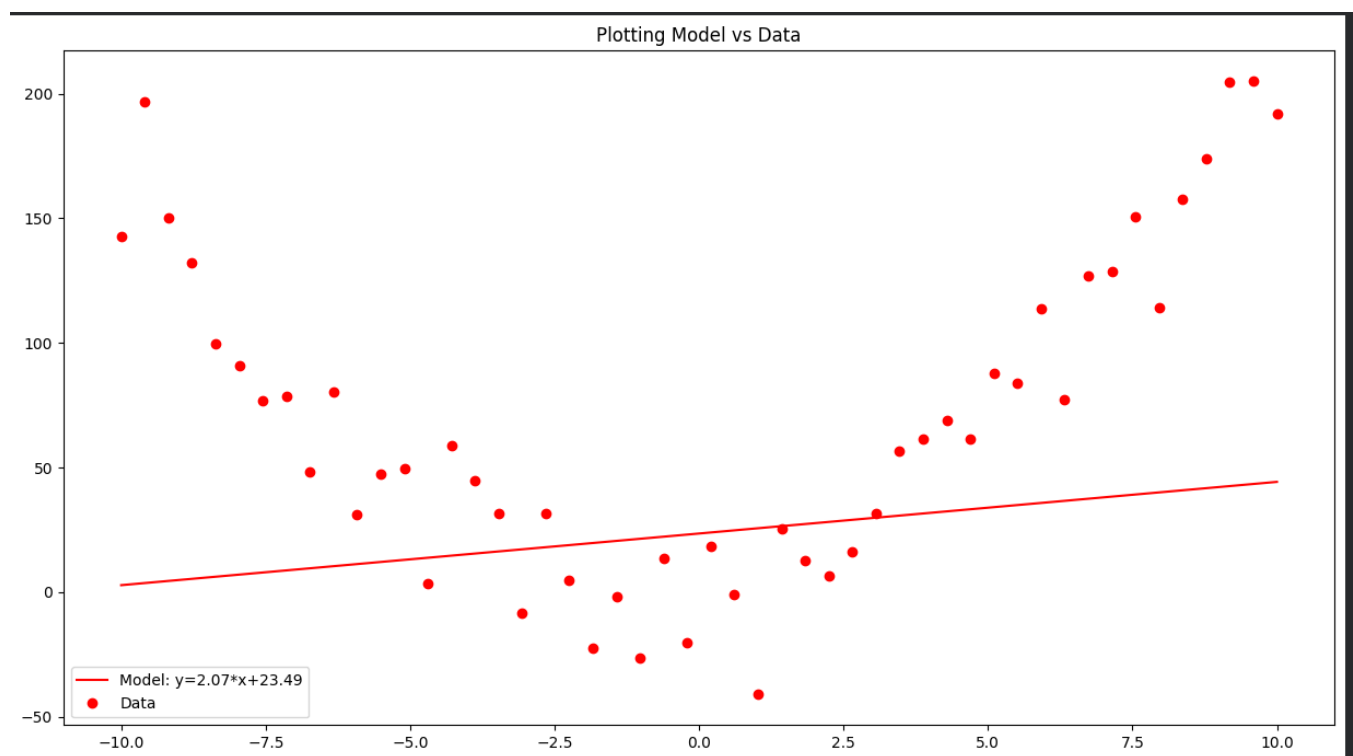
```
predecir = model.predict(x)
```

2/2 ————— 0s 41ms/step

Vamos a configurar la gráfica comparativa final para evaluar visualmente el desempeño del perceptrón. Primero, se traza una línea continua (variable predecir) que representa la solución propuesta por el modelo, y en su etiqueta se inserta dinámicamente la ecuación matemática exacta que aprendió (los valores de peso y sesgo). Simultáneamente, se vuelven a dibujar los datos originales como puntos dispersos para contrastarlos con esa línea. Finalmente, se añaden el título y la leyenda para identificar cada elemento antes de proyectar la imagen definitiva en pantalla.

```
plt.figure(figsize=(15,8))
plt.plot(x, predecir, 'r-', label='Model: y={:.2f}*x+{:.2f}'.format(w[0].item(),w[1].item()))
plt.plot(x, y, 'o', label='Data', color="red")
plt.title('Plotting Model vs Data')
plt.legend(loc=0)
plt.show()
```

En esta gráfica final observamos el resultado del aprendizaje contrastado con la realidad. Los puntos rojos dibujan claramente una curva parabólica (en forma de "U"), mientras que la línea roja continua representa la solución que encontró el perceptrón: una recta con la ecuación $y=2.07x + 23.49$. Aquí se hace evidente la limitación del modelo lineal simple, ya que, por su naturaleza, el perceptrón (tal como está configurado) solo puede trazar líneas rectas y no tiene la capacidad de doblarse para seguir la curvatura de los datos. Por eso, aunque la línea intenta pasar por el centro para reducir el error general, no logra ajustarse correctamente a la forma real del conjunto de datos.



FUNCIÓN DE ACTIVACIÓN POR DEFECTO

Dado que en el código de la capa Dense no especificamos ningún parámetro de activación, Keras utiliza por defecto la activación Lineal (técnicamente llamada linear o identidad).

Comportamiento:

Esta función es la más simple de todas: no transforma la salida. Simplemente toma el resultado de la operación matemática de la neurona y lo entrega tal cual. A diferencia de otras funciones que restringen los valores entre 0 y 1 (como la Sigmoide) o eliminan los negativos (como ReLU), la activación lineal permite que el modelo prediga cualquier valor numérico real (positivo, negativo o cero). Esto es ideal y necesario para tareas de regresión como esta, donde queremos predecir un valor continuo sin límites preestablecidos.

CONCLUSIONES

Aguirre Lanto Víctor Manuel

En esta práctica pudimos comprobar el funcionamiento básico de un perceptrón simple utilizando la librería Keras. Lo más destacable fue observar gráficamente cómo el modelo logró reducir el error progresivamente durante las épocas de entrenamiento, convergiendo finalmente en una línea recta. Sin embargo, al contrastar esta línea con nuestros datos de origen cuadrático, se hizo evidente que una arquitectura lineal y de una sola neurona no es suficiente para capturar patrones curvos complejos, resultando en una aproximación general pero no exacta.

Gasca Fragoso Pedro

A través del desarrollo de este ejercicio, experimentamos con la generación de datos sintéticos añadiendo ruido aleatorio para simular condiciones más realistas. La conclusión principal es que, aunque el modelo aprendió correctamente los parámetros de peso y sesgo minimizando la función de pérdida, su naturaleza lineal le impide adaptarse a la curvatura parabólica de los datos. Esto demuestra las limitaciones teóricas de usar una función de activación lineal por defecto cuando se trabaja con conjuntos de datos que tienen una distribución no lineal.

Guevara Badillo Areli Alejandra

Se analizó la importancia de la configuración de la capa densa en las redes neuronales. Al trabajar en una tarea de regresión y utilizar la activación lineal (por defecto), el modelo tuvo la libertad de predecir valores continuos sin restricciones de rango. No obstante, el resultado visual confirma que para representar adecuadamente la forma de "U" de nuestros datos, el modelo actual es demasiado simple; para ajustar mejor la curva, sería necesario explorar arquitecturas más profundas o diferentes técnicas de procesamiento.

Montiel Toro Arael de Jesús

Esta práctica resaltó la importancia de no confiar únicamente en las métricas numéricas durante el entrenamiento. Aunque observamos que el valor de la pérdida (loss) disminuyó significativamente tras las 10 iteraciones, la visualización final reveló un claro subajuste de los datos. Esto nos enseña que el perceptrón simple busca matemáticamente la mejor línea recta posible para reducir el error promedio, pero carece de la flexibilidad geométrica necesaria para resolver problemas con comportamientos polinómicos.

Ramírez Lozano Gael Martin

La implementación del modelo secuencial con una única entrada y salida nos permitió entender el flujo de trabajo completo en TensorFlow, desde la definición hasta la predicción. Como conclusión, determinamos que el perceptrón simple es una herramienta eficaz para encontrar tendencias lineales generales, pero ante datos con una estructura más compleja (como la función cuadrática generada), el modelo llega a su límite de capacidad. Esto sugiere que para futuros ejercicios con datos curvos, se requerirán redes neuronales con más capas u otras funciones de activación.